

Two-Axis Thrust Vector Control Gimbal Demonstrator

Design, Analysis, Fabrication, and Closed-Loop Control

Michael Sell

September 5, 2026

Abstract

This report documents the design, analysis, and construction of a two-axis thrust vector control (TVC) gimbal demonstrator, inspired by the actuated engine mounts used on SpaceX's Raptor engines. The system uses a nested-ring mechanism, driven by two servo linkages, to actively stabilize an inner tube against disturbance. The loop closes with a 6-axis IMU, a complementary filter, and a PID controller running on an ESP32.



Contents

1	Introduction & Conceptual Design	4
1.1	Motivation	4
1.2	Design Inspiration: Raptor TVC	4
1.3	Initial Sketches	4
1.4	Design Targets	5
2	Constraint & Clearance Analysis	5
2.1	Geometry Setup	6
2.2	Two-Triangle Derivation, Full-Tip Case	6
2.3	Short-Ring Regime, Full Derivation	7
2.4	Worked Calculation: Inner Ring to Outer Ring	8
2.5	Worked Calculation: Outer Ring to Frame	8
2.6	Confirmed Clearances	9
2.7	Demo Stand Tilt Clearance	9
2.8	Linkage Ratio Linearity Check	9
3	Torque Budget & Actuator Sizing	10
3.1	Governing Equation	10
3.2	Real Mass Properties	11
3.3	Worked Calculation, Pitch Axis	11
3.4	Worked Calculation, Yaw Axis	12
3.5	Summary	12
4	CAD & Drawing Package	13
4.1	Parametric Modeling Approach	13
4.2	Sheet Metal Legs and Servo Brackets	14
4.3	GD&T Approach	14
4.4	Drawing Package and Iteration	15
5	Fabrication	16
5.1	Material Selection: PVC & Hand Fabrication Constraints	16
6	Electronics & Sensor Fusion	17
6.1	System Architecture	17
6.2	Complementary Filter	17
6.3	Sensor Calibration	17
6.4	Axis Mapping	17
7	Closed-Loop Control	18
7.1	PID Control: Brief Principles	18
7.2	Anti-Windup	18
7.3	Safety Limit	18
7.4	Final Gains	19
7.5	Pitch/Yaw Divergence: A Late-Stage Debugging Story	19
8	WiFi Live-Tuning Dashboard	20

9	Circuit Design & PCB	21
9.1	Switching Circuit Validation	21
9.2	PCB / Carrier Board Approach	21
10	Results	22
10.1	Step Response: Isolated Axes	23
10.2	Step Response: Combined Two-Axis Disturbance	24
10.3	Methodology and Worked Calculation	25
10.4	Per-Event Results	26
10.5	Quantitative Summary	27
10.6	Before and After the Sign Correction	27
11	Iteration & Lessons Learned	28
11.1	Named Future Improvements	29
12	Conclusion	29
13	Bill of Materials	30
A	Full Drawing Package	30
B	Firmware Listing	31

1 Introduction & Conceptual Design

1.1 Motivation

The inspiration for this project ultimately stems from the desire to learn. This, combined with the desire to showcase my engineering skills in a visually compelling portfolio piece, led to the decision to build a two-axis thrust vector control (TVC) gimbal demonstrator. The main inspiration for this project came from the engine mounts and TVC systems used on SpaceX's Raptor engines after seeing the complexities involved and being fascinated by the technology. Not only was this project relevant to recent professional endeavors, but it was interesting to me personally and it was something that I had very little prior knowledge about.

The goal of the project was to showcase as broad of a range of engineering skills as possible. This meant that the project would need to include conception, mechanical design, electrical design, control systems, and software development. The combination of these skills allows for the demonstration of a complete system, from concept to implementation. Along with this, the project was also designed to be visually compelling and hopefully inspire others to continue learning and exploring the world of engineering.

1.2 Design Inspiration: Raptor TVC

The mechanism design draws directly from the actuated engine mounts used on SpaceX's Raptor engines (albeit a different mechanism), which gimbal the entire engine assembly to steer thrust vector direction rather than relying on separate control surfaces.

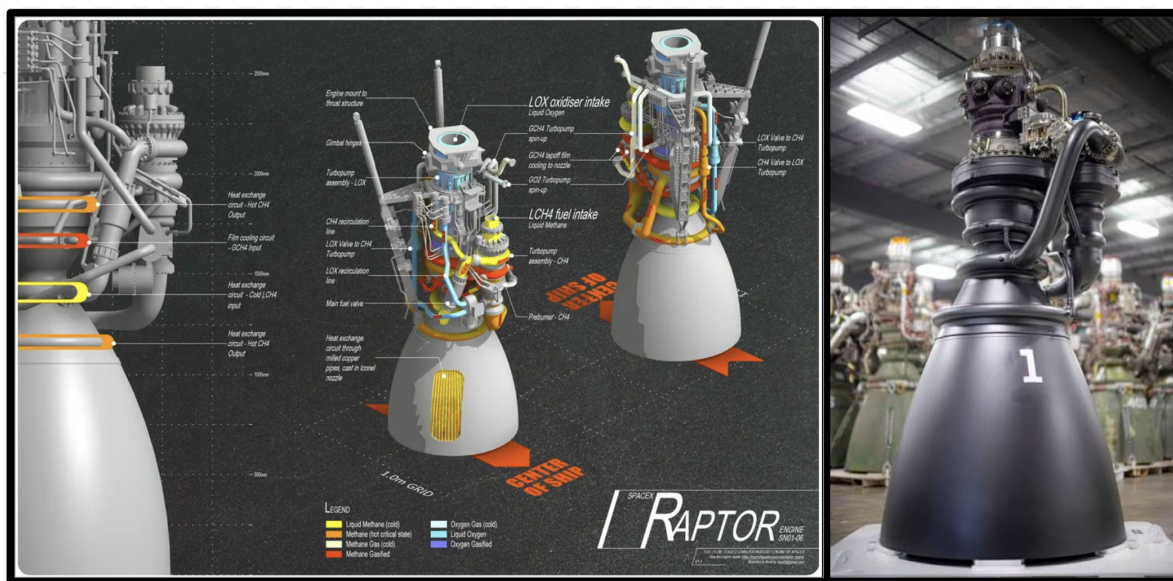


Figure 1: SpaceX Raptor engine [1], the inspiration for this project's actuated gimbal mechanism.

1.3 Initial Sketches

Initial conception began with hand sketches, exploring the nested-ring mechanism, linkage geometry, and overall layout. These sketches are largely conceptual and clearly do not resemble the final project geometry. However, they were essential in the brainstorming phase and helped to establish the overall design direction.

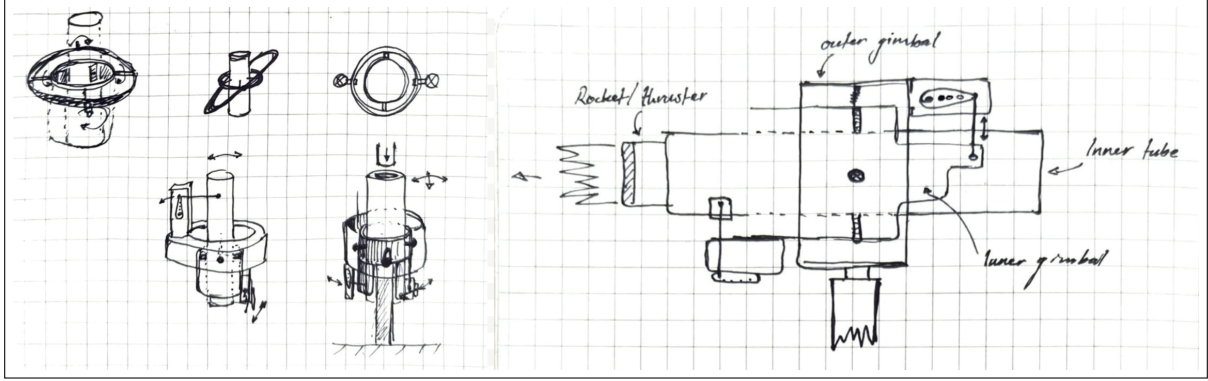


Figure 2: Initial concept page, July 7, 2026.

1.4 Design Targets

The following constraints were fixed at the outset of the project and used throughout all subsequent clearance and torque calculations:

- Per-axis deflection: $\theta = 15^\circ$
- Combined worst-case deflection (both axes simultaneously): $\theta_c = \sqrt{2} \times 15^\circ = 21.21^\circ$
- Fabrication margin: 3 on all clearance checks
- Linkage ratio: 2:1 (servo rotation : gimbal deflection)

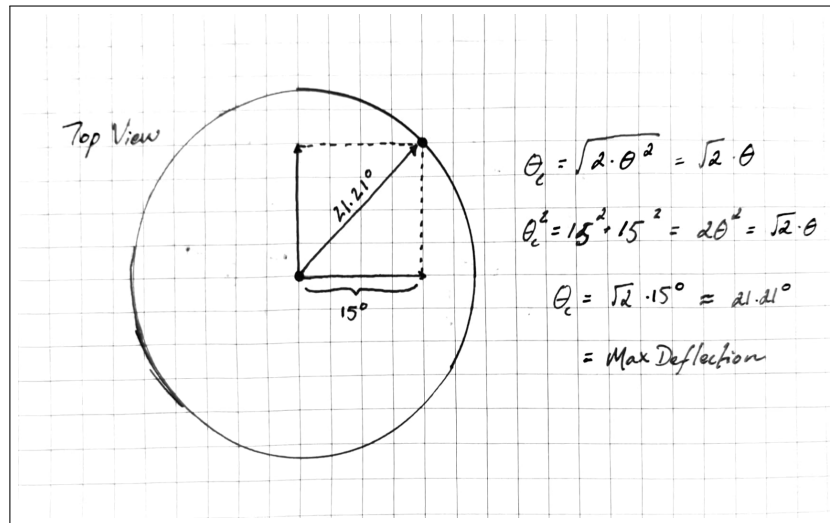


Figure 3: Combined-deflection derivation for $\theta = 15^\circ$.

2 Constraint & Clearance Analysis

This section derives, from first principles, the geometric clearance required between each pair of nested rings as they rotate through their full range of motion.

2.1 Geometry Setup

Each ring pair is modeled as a rigid arm pivoting about a fixed point, decomposed into two perpendicular legs: an axial offset h and a radial offset r . Rotating this arm by angle θ requires resolving both legs' contributions to the resulting radial reach independently.

2.2 Two-Triangle Derivation, Full-Tip Case

Consider a ring pivoting about a fixed point, with radius r_2 (its outer edge) and half-height h (more specifically, its axial extent from the pivot to its end). At rest, the straight-line distance from the pivot to the ring's farthest corner is the hypotenuse of a right triangle with legs r_2 and h :

$$L_e = \sqrt{r_2^2 + h^2} \quad (1)$$

This hypotenuse sits at a fixed angle φ from the axial (vertical) direction, determined entirely by the ring's own proportions (this is the furthest radial reach the ring's corner can achieve, hence the h being 1/2 the ring's total height):

$$\varphi = \arctan\left(\frac{r_2}{h}\right), \quad \sin \varphi = \frac{r_2}{L_e}, \quad \cos \varphi = \frac{h}{L_e} \quad (2)$$

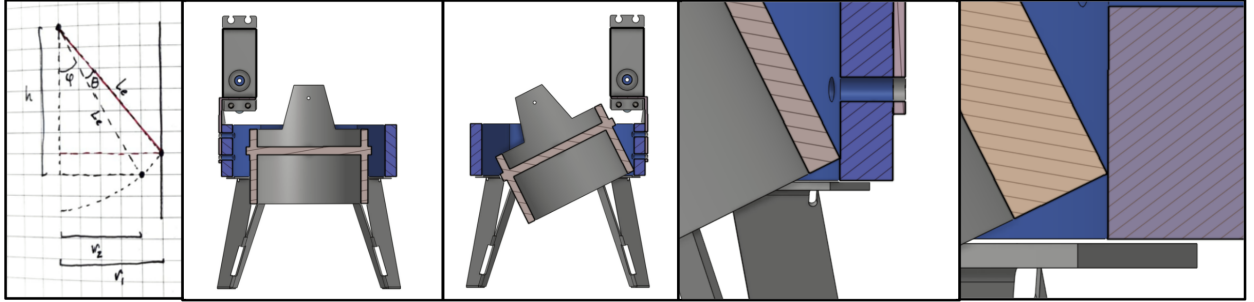


Figure 4: Full-tip geometry: pivot, radius r_2 , half-height h , hypotenuse L_e at rest angle φ , rotated by θ .

When the ring rotates by the design deflection angle θ , the corner's hypotenuse now sits at angle $\varphi + \theta$ from the axial direction (red traced hypotenuse). The radial distance this corner reaches from the pivot axis is:

$$r_1 = L_e \sin(\varphi + \theta) \quad (3)$$

Expanding with the angle-sum identity:

$$\sin(\varphi + \theta) = \sin \varphi \cos \theta + \cos \varphi \sin \theta \quad (4)$$

Substituting the expressions for $\sin \varphi$ and $\cos \varphi$ found above:

$$r_1 = L_e \left[\frac{r_2}{L_e} \cos \theta + \frac{h}{L_e} \sin \theta \right] \quad (5)$$

The L_e terms cancel cleanly, leaving a result that depends only on the ring's own radius and half-height, not on the hypotenuse itself:

$$r_1 \geq r_2 \cos \theta + h \sin \theta + \text{margin} \quad (6)$$

This is the full-tip clearance requirement: it gives the minimum inner radius a containing ring must have for a rotating ring's farthest corner to clear it at deflection θ .

2.3 Short-Ring Regime, Full Derivation

If the containing ring is short enough that the rotating ring's corner sweeps past its edge before reaching full radial extension, the full-tip formula above no longer applies. Instead, the constraint becomes: the containing ring's own corner, swung *backward* by θ , must just clear the rotating ring's surface at radius r_2 .

Set up the same way, but now using the containing ring's own dimensions r_{out} and h_{out} :

$$L_w = \sqrt{r_{\text{out}}^2 + h_{\text{out}}^2}, \quad \varphi_w = \arctan\left(\frac{r_{\text{out}}}{h_{\text{out}}}\right), \quad \sin \varphi_w = \frac{r_{\text{out}}}{L_w}, \quad \cos \varphi_w = \frac{h_{\text{out}}}{L_w} \quad (7)$$

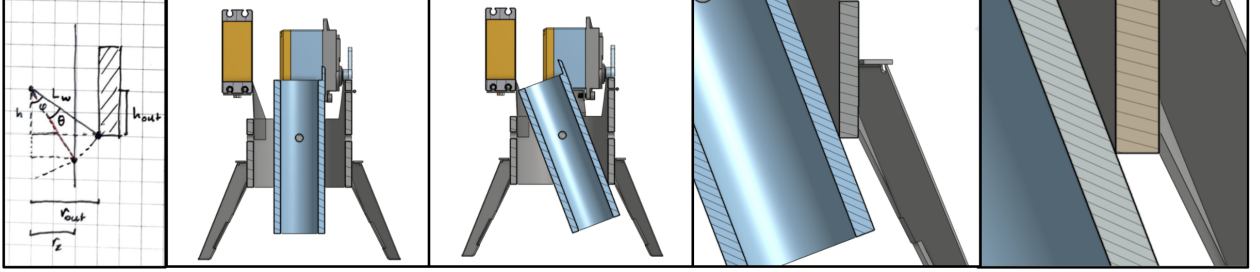


Figure 5: Short-ring geometry: hypotenuse L_w from the containing ring's own corner, rest angle φ_w , rotated backward by θ .

The governing condition is that this corner, rotated backward by θ , lands exactly at r_2 :

$$L_w \sin(\varphi_w - \theta) = r_2 \quad (8)$$

Expanding:

$$\sin(\varphi_w - \theta) = \sin \varphi_w \cos \theta - \cos \varphi_w \sin \theta \quad (9)$$

Substituting:

$$r_2 = L_w \left[\frac{r_{\text{out}}}{L_w} \cos \theta - \frac{h_{\text{out}}}{L_w} \sin \theta \right] = r_{\text{out}} \cos \theta - h_{\text{out}} \sin \theta \quad (10)$$

Solving for r_{out} :

$$r_{\text{out}} \cos \theta = r_2 + h_{\text{out}} \sin \theta \quad (11)$$

$$r_{\text{out}} \geq \frac{r_2}{\cos \theta} + h_{\text{out}} \tan \theta + \text{margin} \quad (12)$$

This gives the minimum radius a *short* containing ring needs, correctly accounting for the fact that the rotating ring's corner passes the containing ring's edge before reaching its full theoretical radial extent.

The calculator automatically selects between Equation 6 and Equation 12 based on each ring pair's actual proportions.

2.4 Worked Calculation: Inner Ring to Outer Ring

Inputs: inner ring OD = 33.47 mm (1" nominal, measured), so $r_2 = 16.735$ mm. Inner ring half-height $h = 50$ mm. Deflection $\theta = 15^\circ$: this interface uses the single-axis pitch deflection, not the combined worst case, since yaw rotates the outer ring and everything inside it (including the inner tube) together as a single unit and creates no relative tilt between inner tube and outer ring. Margin = 3 mm.

Step 1, naive full-tip estimate. Applying Equation 6 directly, before checking which regime actually governs:

$$r_{1,\text{naive}} = r_2 \cos \theta + h \sin \theta + \text{margin} = 16.735(0.9659) + 50(0.2588) + 3 = 32.11 \text{ mm} \quad (13)$$

Step 2, check the actual regime. Compute L_e and φ :

$$L_e = \sqrt{r_2^2 + h^2} = \sqrt{16.735^2 + 50^2} = 52.73 \text{ mm}, \quad \varphi = \arctan\left(\frac{r_2}{h}\right) = 18.51^\circ \quad (14)$$

The real outer ring stock's half-height, $h_{\text{outer}} = 30$ mm, is shorter than the inner ring's own half-height (50 mm). This means the inner ring's tip, when swung by θ , clears past the outer ring's own end before reaching full radial extension, so the *short-ring* regime (Equation 12) governs instead of the naive full-tip case.

Step 3, correctly-regime-selected requirement.

$$r_{\text{outer}} \geq \frac{r_2}{\cos \theta} + h_{\text{outer}} \tan \theta + \text{margin} = \frac{16.735}{0.9659} + 30(0.2679) + 3 = 28.36 \text{ mm} \quad (15)$$

Step 4, check against real stock. The selected outer ring (2-1/2" nominal coupling) has a measured ID of 69.06 mm, giving $r_{1,\text{actual}} = 30.325$ mm:

$$\text{Clearance} = 30.325 - 28.36 = 1.96 \text{ mm} \quad \checkmark \text{ clears} \quad (16)$$

2.5 Worked Calculation: Outer Ring to Frame

Inputs: outer ring's own radius $r_{2,\text{outer}} = 34.53$ mm, half-height $h_{\text{outer}} = 30$ mm. Frame half-height $h_{\text{frame}} = 15$ mm. Deflection $\theta = 15^\circ$: this interface uses the single-axis yaw deflection, since pitch is internal to the outer ring and does not tilt the outer ring relative to the frame at all.

Step 1, check regime. The frame ($h_{\text{frame}} = 15$ mm) is now shorter than the outer ring's own half-height ($h_{\text{outer}} = 30$ mm) - the opposite situation from the full-tip case. The short-ring regime governs here.

Step 2, radial requirement.

$$r_{\text{frame}} \geq \frac{r_{2,\text{outer}}}{\cos \theta} + h_{\text{frame}} \tan \theta + \text{margin} = \frac{34.53}{0.9659} + 15(0.2679) + 3 = 42.77 \text{ mm} \quad (17)$$

Frame's actual radius (from a measured 44.63 mm internal socket dimension) gives:

$$\text{Clearance} = 44.63 - 42.77 = 1.86 \text{ mm} \quad \checkmark \text{ clears} \quad (18)$$

Axial check: not applicable. With $h_{\text{frame}} = 15$ mm now shorter than $h_{\text{outer}} = 30$ mm, the frame no longer axially encloses the outer ring's full extent even at rest - it functions as a radial support band around the tube's middle rather than a full enclosure. The axial-enclosure check used when the frame was the taller member no longer applies to this geometry and is omitted.

2.6 Confirmed Clearances

Table 1: Ring-pair clearance results, each interface at its own governing single-axis deflection ($\theta = 15^\circ$)

Interface	Required (mm)	Actual (mm)	Margin (mm)	Governing regime
Inner ring $\rightarrow$ Outer ring (radial)	28.36	30.325	1.96	Short-ring
Outer ring $\rightarrow$ Frame (radial)	42.77	44.63	1.86	Short-ring

2.7 Demo Stand Tilt Clearance

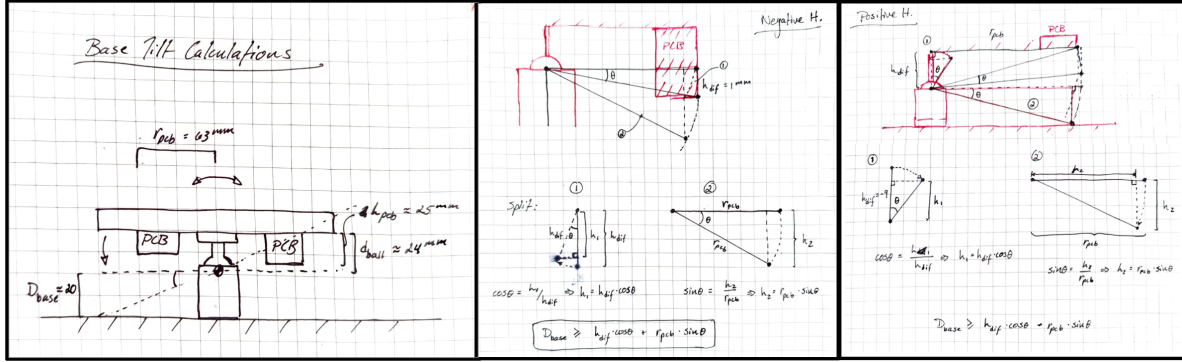


Figure 6: Demo stand tilt clearance geometry, showing the electronics assembly's lowest point h_{pcb} relative to the ball-head pivot d_{ball} , and the required vertical clearance D_{base} to the stationary base plate. Right two images showing demo stand tilt calculations where the difference in height between the base plate and pcb is negative, and positive, respectively.

The same two-triangle method was reused to size the demo stand's legs, this time solving for the vertical clearance D_{base} required between the ball-head pivot and the stationary base plate as the whole assembly tilts by θ_{demo} :

$$D_{base} \geq v \cos \theta_{demo} + r_{pcb} \sin \theta_{demo} + \text{margin} \quad (19)$$

where $v = d_{ball} - h_{pcb}$ is the signed vertical offset of the electronics assembly's lowest point relative to the pivot. Two populated PCBs were checked independently; the governing (larger) required clearance determined final leg length. Along with mathematical calculations, a simulated check was done by modeling this and in Onshape and visually confirming by using mate features that the assembly could tilt to the required angle without interference.

2.8 Linkage Ratio Linearity Check

The 2:1 servo-to-gimbal linkage ratio assumes the servo arm's small angular sweep translates linearly into gimbal deflection. This assumption was checked directly by comparing arc length against chord length at the design deflection of $\theta = 15^\circ$:

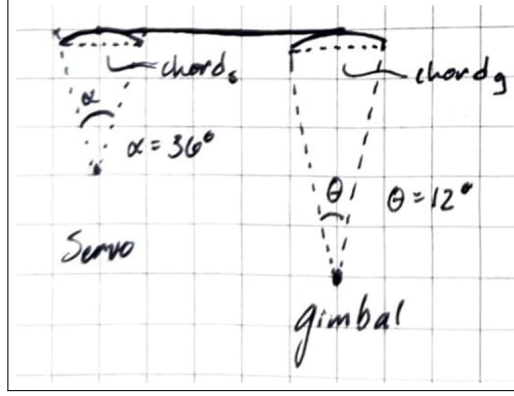


Figure 7: Sketch of linkage ratio geometry, where theta is 12 instead of 15 in this case (previous constraint).

Gimbal side (independent of linkage ratio):

$$\text{Arc: } S_a = 0.26180r \quad (20)$$

$$\text{Chord: } S_c = 2r \sin(7.5^\circ) = 0.26105r \quad (21)$$

$$\text{Error} = \frac{S_a - S_c}{S_a} \approx 0.29\% \quad (22)$$

Servo side, at the actual 2:1 ratio (servo angle = $2 \times 15^\circ = 30^\circ$):

$$\text{Arc: } S_a = 0.52360r \quad (23)$$

$$\text{Chord: } S_c = 2r \sin(15^\circ) = 0.51764r \quad (24)$$

$$\text{Error} = \frac{S_a - S_c}{S_a} \approx 1.14\% \quad (25)$$

Both errors are small enough that the linear small-angle approximation was adopted for the linkage geometry without further correction, particularly given the system operates closed-loop, where any small residual geometric nonlinearity is continuously corrected by the PID controller rather than accumulating open-loop.

3 Torque Budget & Actuator Sizing

3.1 Governing Equation

Each axis's required torque was budgeted from three contributions:

$$\tau_{\text{gravity}} = mg d_{\text{cg}} \sin \theta \quad (26)$$

$$\tau_{\text{thrust}} = FL \sin \theta \quad (27)$$

$$\tau_{\text{inertial}} = I\alpha, \quad I = m d_{\text{cg}}^2, \quad \alpha = \frac{4\theta_{\text{rad}}}{t^2} \quad (28)$$

with a 1.30 friction margin applied and divided by the 2:1 linkage ratio to reach the required servo torque:

$$\tau_{\text{servo}} = \frac{1.30(\tau_{\text{gravity}} + \tau_{\text{thrust}} + \tau_{\text{inertial}})}{2} \quad (29)$$

3.2 Real Mass Properties

Mass properties were pulled directly from Onshape (not estimated), using mate-connector-referenced centers of mass for each axis's actual moving assembly. Distance from pivot to center of mass, d_{cg} , was computed directly from the three-axis offset reported by Onshape:

$$d_{cg} = \sqrt{X^2 + Y^2 + Z^2} \quad (30)$$

Pitch (inner tube assembly): $X = 0.364$ mm, $Y = 0.44$ mm, $Z = -7.144$ mm, mass = 66.582 g

$$d_{cg,pitch} = \sqrt{0.364^2 + 0.44^2 + 7.144^2} = 7.167 \text{ mm} = 0.007167 \text{ m} \quad (31)$$

Yaw (outer tube assembly): $X = -2.948$ mm, $Y = -14.721$ mm, $Z = 14.363$ mm, mass = 206.125 g

$$d_{cg,yaw} = \sqrt{2.948^2 + 14.721^2 + 14.363^2} = 20.777 \text{ mm} = 0.020777 \text{ m} \quad (32)$$

Other inputs, held constant across both axes: thrust reaction force $F = 0.15$ N (fan), moment arm to fan $L = 0.038$ m (measured directly from the real pivot-to-fan geometry, 101 mm total minus 63 mm pivot-to-tube-base), target settle time $t = 0.2$ s, friction margin 30%, deflection $\theta = 15^\circ = 0.2618$ rad.

3.3 Worked Calculation, Pitch Axis

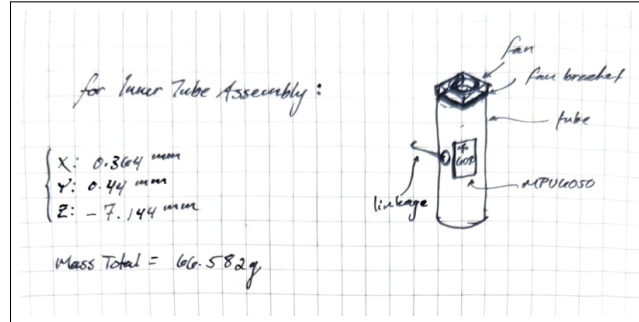


Figure 8: Pitch axis torque budget sketch, showing center of gravity.

$$\tau_{gravity} = m g d_{cg} \sin \theta = 0.066582 \times 9.81 \times 0.007167 \times \sin(15^\circ) = 1.212 \times 10^{-3} \quad (33)$$

$$\tau_{thrust} = F L \sin \theta = 0.15 \times 0.038 \times \sin(15^\circ) = 1.475 \times 10^{-3} \quad (34)$$

$$I = m d_{cg}^2 = 0.066582 \times (0.007167)^2 = 3.421 \times 10^{-6} \quad (35)$$

$$\alpha = \frac{4\theta}{t^2} = \frac{4 \times 0.2618}{(0.2)^2} = 26.18 \quad (36)$$

$$\tau_{inertial} = I \alpha = 3.421 \times 10^{-6} \times 26.18 = 8.96 \times 10^{-5} \quad (37)$$

$$\tau_{subtotal} = 0.001212 + 0.001475 + 0.0000896 = 0.002777 \text{ N}\cdot\text{m} \quad (38)$$

$$\tau_{total} = 1.30 \times 0.002777 = 0.003610 \text{ N}\cdot\text{m} \quad (39)$$

$$\tau_{\text{servo}} = \frac{0.003610}{2} = 0.001805 \text{ N}\cdot\text{m} \quad (40)$$

$$\text{Safety factor} = \frac{0.92}{0.001805} \approx 510 \times \quad (41)$$

3.4 Worked Calculation, Yaw Axis

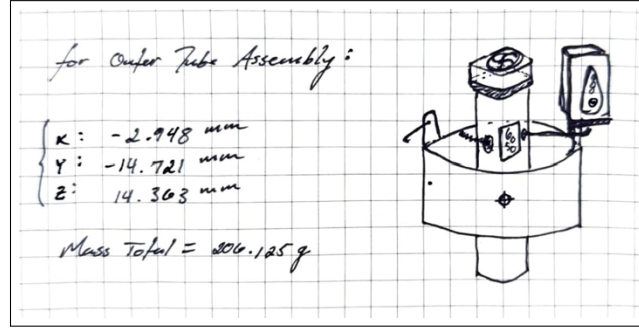


Figure 9: Yaw axis torque budget sketch, showing center of gravity.

$$\tau_{\text{gravity}} = m g d_{\text{cg}} \sin \theta = 0.206125 \times 9.81 \times 0.020777 \times \sin(15^\circ) = 1.0875 \times 10^{-2} \quad (42)$$

$$\tau_{\text{thrust}} = F L \sin \theta = 0.15 \times 0.038 \times \sin(15^\circ) = 1.475 \times 10^{-3} \quad (\text{same } F, L, \theta \text{ as pitch}) \quad (43)$$

$$I = m d_{\text{cg}}^2 = 0.206125 \times (0.020777)^2 = 8.899 \times 10^{-5} \quad (44)$$

$$\alpha = \frac{4\theta}{t^2} = 26.18 \quad (\text{same } \theta, t \text{ as pitch}) \quad (45)$$

$$\tau_{\text{inertial}} = I \alpha = 8.899 \times 10^{-5} \times 26.18 = 2.330 \times 10^{-3} \quad (46)$$

$$\tau_{\text{subtotal}} = 0.010875 + 0.001475 + 0.002330 = 0.014680 \text{ N}\cdot\text{m} \quad (47)$$

$$\tau_{\text{total}} = 1.30 \times 0.014680 = 0.019084 \text{ N}\cdot\text{m} \quad (48)$$

$$\tau_{\text{servo}} = \frac{0.019084}{2} = 0.009542 \text{ N}\cdot\text{m} \quad (49)$$

$$\text{Safety factor} = \frac{0.92}{0.009542} \approx 96 \times \quad (50)$$

3.5 Summary

Both axes carry substantial margin against the selected MG996R servo's rated torque (0.92 at 4.8), an intentional choice given the servo's low incremental cost relative to the risk of an undersized actuator.

Table 2: Torque budget results, both axes

Axis	Mass (kg)	d_{cg} (m)	Safety Factor
Pitch	0.066582	0.007167	$\approx 510\times$
Yaw	0.206125	0.020777	$\approx 96\times$

4 CAD & Drawing Package

4.1 Parametric Modeling Approach

For this project, the online software Onshape was chosen to parametrically model the design. This would allow for easy modification and iteration of the design. Specifically, key dimensions (leg length, mounting offsets) were maintained as named Onshape variables rather than hard-coded values, allowing late-stage geometry changes (e.g. leg length, once real PCB height was known) to propagate automatically rather than requiring manual rework. Using this software also facilitated the creation of a full drawing package, including a comprehensive bill of materials (BOM) and revision table, which was useful in tracking changes throughout the project. The full assembly is visible in Figure 10 and is publicly available at the following url: <https://cad.onshape.com/documents/38454ed41460ebb3f5376222/w/7e6b36a949ced50e5d7d3c83/e/bf2871f1439507b37564f52c>

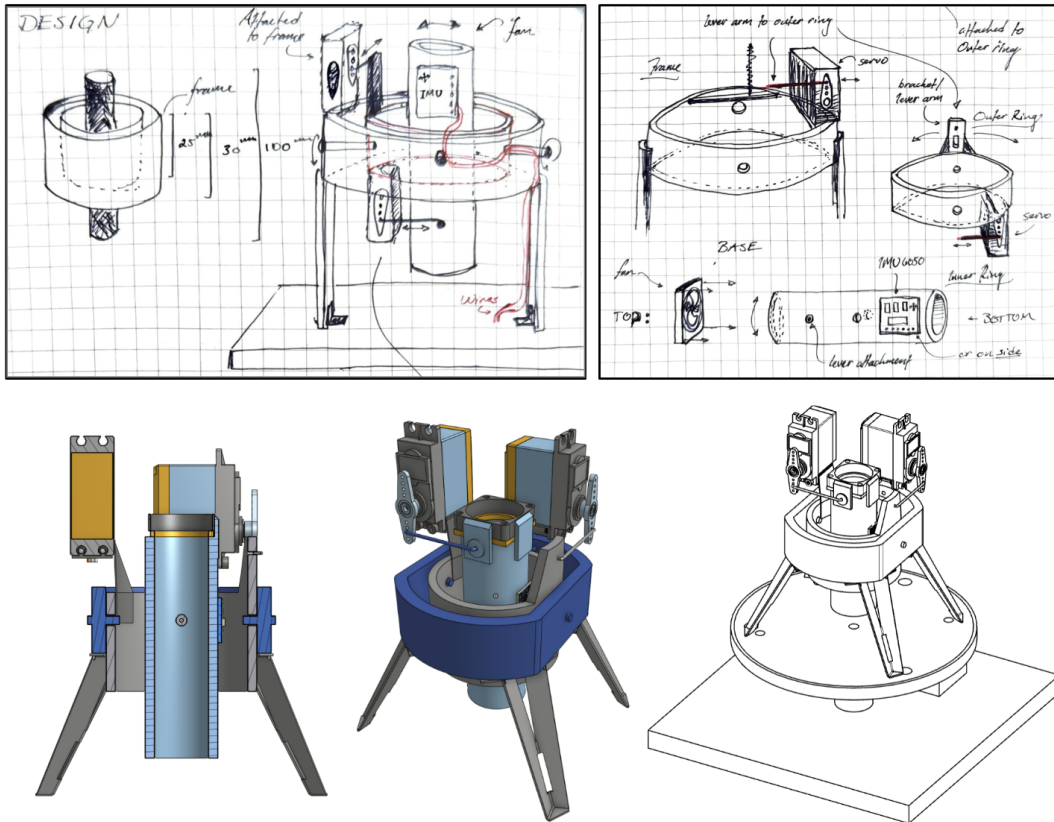


Figure 10: Full assembly design sketch, showing frame, outer ring, base, and wiring routing with corresponding cad model and drawing.

4.2 Sheet Metal Legs and Servo Brackets

The demonstration stand's legs combine a purchased telescoping antenna (upper section) with a hand-formed aluminum V-channel (lower section), modeled in Onshape's Sheet Metal workflow to generate an accurate flat pattern for hand fabrication (Figure 11A-D). The servo brackets were also modeled in Onshape but, because of the complex double axis bend, the software was not able to generate a true sheet metal pattern. Therefore, an alternate method was used instead: a pen and ruler (Figure 11E-F). All sheet metal parts were fabricated from 0.8mm brushed aluminum sheet stock and were formed using a combination of hand tools and a small bench vise.

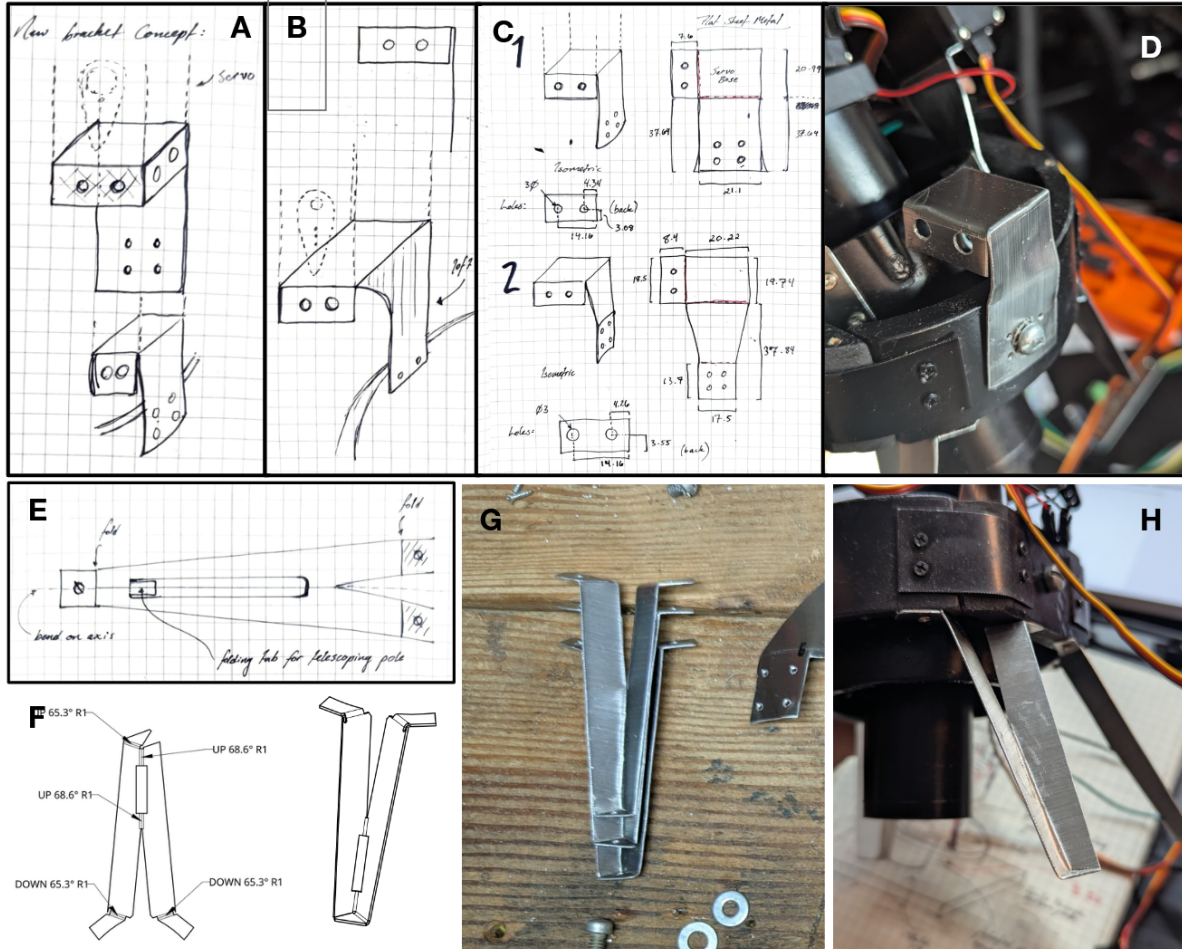


Figure 11: A-D) Sheet metal servo bracket design sketches, templates, and hand-fabricated implementation. E-F) Sheet metal leg design sketches, templates, and hand-fabricated implementation.

4.3 GD&T Approach

Given hand fabrication (not machined production), full GD&T was deliberately not applied. Instead, the following tolerances were noted:

- Bend angles: $\pm 2^\circ$ tolerance, called out explicitly given bending is the least repeatable hand operation in the build.
- Linear dimensions: ± 0.5 mm general tolerance.

4.4 Drawing Package and Iteration

Figure 10 depicts the process of creating the full assembly design sketch, showing the initial conceptualizations of features such as the frame geometry, servo placement, and imu placement. However, the final design was iterated upon and refined from the conceptual phase. Namely, the frame was flattened on either side so as to avoid the requirement for spacers and to allow for a more compact and structurally sound design. The servo placement was also iterated upon and it can be seen in the figure that both servos were moved to the same side of the assembly as opposed to opposing sides as seen in the conceptual sketches. This was done to allow for a more compact design overall, as the bottom mounted servo would have required a larger gap between the frame and the base plate due to the extension past the lowest part of the inner tube.

The sensor placement was also changed from the conceptual sketches, as the IMU unit was moved to the center of the inner tube. This was done in hopes to eliminate any potential gravitational bias that may be introduced by the IMU being mounted outside of the pivot point. In theory, in this configuration, the IMU bias on the yaw axis should be negligible and the pitch axis bias is minimized for a mounting configuration that isn't inside of the inner tube. Most of the iteration progress described above can be seen in Figure 12.

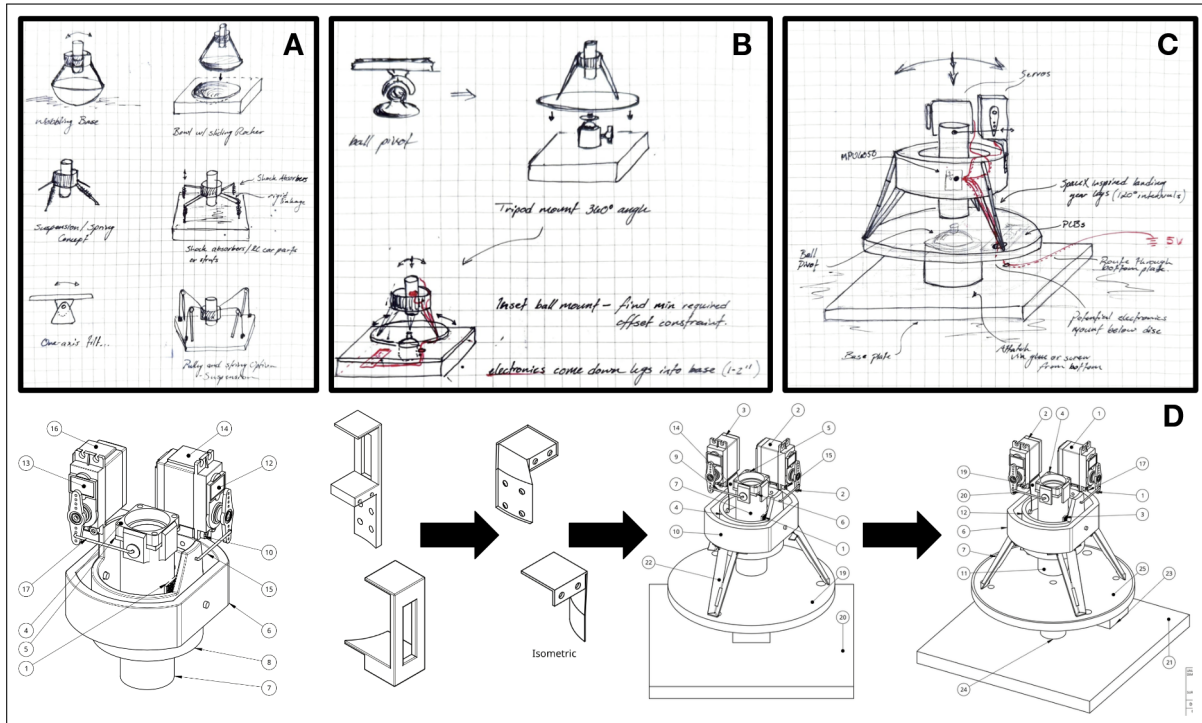


Figure 12: A) Conceptual base plate design sketches; B) Ball point pivot for base plate concept; C) Full base plate concept V.01 incorporating a base plate and connected ball point pivot joint for full range of motion; D) Assembly revision progression, showing evolution of the brackets, legs, and base plate geometry.

5 Fabrication

5.1 Material Selection: PVC & Hand Fabrication Constraints

A self imposed budget of \$100 was set in place at the beginning of the project, which limited material selection to low cost, commercially available materials. After brainstorming, PVC was selected as the primary material for the inner and outer tube assemblies, as well as the frame. PVC was low cost, easy to machine by hand, durable, and readily available in a variety of sizes.

Given no access to a 3D printer or machine shop, all custom parts were designed around this commercially available catalog stock (PVC pipe/couplings, aluminum sheet) and hand tools. This constraint is treated throughout this report as a documented design decision rather than a limitation; see Section 11 for named improvements that would be pursued given access to additional fabrication tools.

The inner tube was selected to be 1" nominal PVC pipe, while the outer tube was selected to be 2-1/2" nominal PVC coupling. The frame was selected to be 3" nominal PVC coupling (Figure 13). The final design was iterated upon to ensure that the inner tube assembly would clear the outer tube assembly, and that the outer tube assembly would clear the frame, at the maximum deflection angle of 15 degrees. The final design also ensured that the inner tube assembly would clear the base plate at the maximum deflection angle of 15 degrees. These calculations can be seen in Section 2

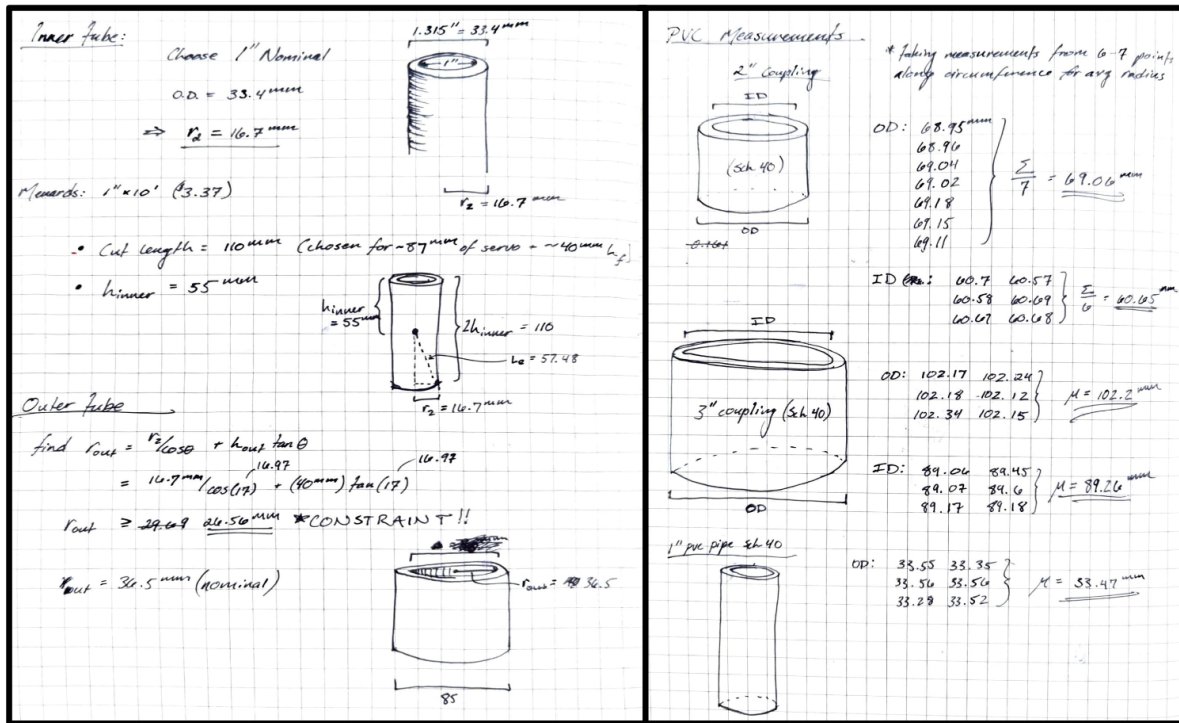


Figure 13: Inner/outer tube sizing derivation using real catalog dimensions and PVC caliper measurements, 1" pipe and 2"/3" couplings, averaged over 6-7 points around each circumference.

6 Electronics & Sensor Fusion

6.1 System Architecture

An ESP32 (WROOM-32D) reads a 6-axis IMU (MPU6050) over I2C, computes a fused orientation estimate, and drives two servos through a PCA9685 PWM driver, also communicating over I2C and sharing the same bus at a different address.

6.2 Complementary Filter

Orientation is estimated by fusing gyroscope integration (fast, drift-prone) with an accelerometer-derived tilt angle (slow, but drift-free), following the standard complementary filter approach [2]:

$$\theta_{\text{fused}} = \alpha(\theta_{\text{fused}} + \dot{\theta}_{\text{gyro}} \Delta t) + (1 - \alpha) \theta_{\text{accel}} \quad (51)$$

with $\alpha = 0.9$ on both axes in the final configuration (see Section 7.5 for why the two axes ultimately use identical filter parameters).

6.3 Sensor Calibration

Six offsets (three accelerometer, three gyroscope) were determined by averaging 300 samples with the sensor held stationary in its final mounted orientation, and re-derived after every physical remount. Final offsets used in the firmware are as follows:

Table 3: Final IMU offset values (calibration averages, final mounted orientation)

Axis	Accelerometer offset (g)	Gyroscope offset (deg/s)
X	-0.9616	0.5768
Y	-0.0555	1.3094
Z	-0.0584	-0.6862

6.4 Axis Mapping

As previously stated, the IMU’s final mounting position is on the side of the inner tube, with the axis orientations as seen in Figure 14. As will be discussed later in the report, there were a number of issues with the two axes’ behavior when tuning the PID values. Full troubleshooting details can be found in the README on the github repo for the project. For now, this gives us a concrete mapping that we can now input the gyroscope and accelerometer data into which will feed into the PID loop and dictate our servo behavior.

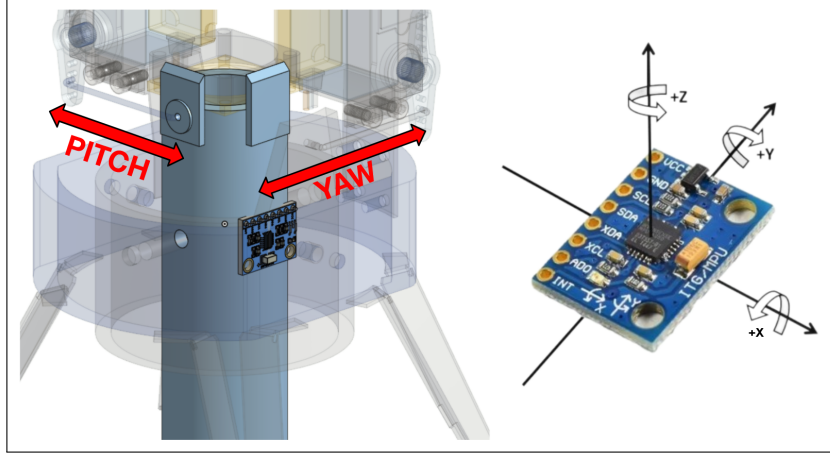


Figure 14: IMU axis mapping sketch, showing the IMU’s physical orientation on the inner tube and the corresponding raw axis mapping to pitch and yaw.

7 Closed-Loop Control

7.1 PID Control: Brief Principles

A standard PID control law was used per axis:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (52)$$

where $e(t)$ is the fused orientation error relative to a level setpoint. The proportional term provides an immediate, error-proportional correction; the integral term eliminates steady-state error under sustained disturbance (a genuinely necessary term for this application, since the stand is tilted and *held* during demonstration, not just briefly disturbed); the derivative term damps oscillatory overshoot.

7.2 Anti-Windup

A conditional-integration anti-windup scheme was implemented: the integral term only accumulates when the commanded output is not already saturated at the mechanical safety limit, or when the current error would pull the output back into range. This was a necessary correction after an earlier, simpler fixed-ceiling integral clamp allowed the controller to remain saturated for extended periods during large disturbances, producing a slow, overshooting recovery once the disturbance was removed.

7.3 Safety Limit

All commanded corrections are hard-clamped to a maximum gimbal deflection of 15° , independent of the calculated PID output. This value is set to match the per-axis deflection target established in Section 2 and used throughout every clearance and torque calculation in this report, rather than being an independently chosen firmware constant. This guarantees the firmware can never command the mechanism beyond the deflection range the physical design was actually validated for. However, while this did prevent any catastrophic movements, this value can be raised to 20° for a more visually striking demonstration.

7.4 Final Gains

Table 4: Final PID gains

Axis	K_p	K_i	K_d
Pitch	-1.0	-15.0	-0.0001
Yaw	-1.0	-15.0	-0.0001

Final complementary filter blend: 0.9 gyro / 0.1 accelerometer on *both* axes. As detailed in Section 7.5, the two axes ultimately run fully identical gains and blend ratios; the earlier belief that pitch required a different configuration turned out to stem from a firmware bug, not a genuine physical difference between the axes.

7.5 Pitch/Yaw Divergence: A Late-Stage Debugging Story

Despite running structurally identical control code on both axes, pitch persistently exhibited behavior yaw never did: jitter at rest, edge oscillation at a naive hard deadband boundary, an integral windup jolt, and a brief wrong-direction servo flick at the onset of any fast tilt. Each of the first three had clean, confirmed root causes: a discontinuous hard deadband (fixed by making the deadband continuous, subtracting the threshold from the error rather than gating output on and off), integral accumulation continuing silently during the deadband freeze (fixed by the same continuous-deadband change, since the error itself now reaches zero rather than being externally gated), and a stale gyroscope offset macro left pointing at an outdated calibration constant after a later recalibration.

The remaining flick proved more stubborn. A first-principles hypothesis was pursued in depth: the IMU is mounted 0.1674m from the pitch pivot but essentially at the yaw pivot, so real centripetal ($l\omega^2$) and tangential ($l\alpha$) acceleration should appear at the sensor during pitch rotation and not during yaw. This effect is well documented in the IMU/gimbal literature and was confirmed as a genuinely real phenomenon. In practice, the centripetal and tangential acceleration terms were computed directly from angular velocity and angular acceleration, then subtracted from the raw accelerometer reading before the tilt angle was calculated:

$$a_c = l\omega^2, \quad a_t = l\alpha, \quad \alpha = \frac{d\omega}{dt} \quad (53)$$

where l is the measured distance from the pivot to the IMU. Both terms were converted to units of g and subtracted from the relevant raw accelerometer axis before computing the corrected tilt angle:

$$a_{x,\text{corr}} = a_x - \frac{l\omega^2}{g}, \quad a_{z,\text{corr}} = a_z - \frac{l\alpha}{g} \quad (54)$$

```

1 float omega = gyroRate * PI / 180.0;
2 float alpha = (omega - lastOmega) / dt;
3 float centripetal_g = (L * omega * omega) / G;
4 float tangential_g = (L * alpha) / G;
5 float axCorrected = ax - centripetal_g;
6 float azCorrected = az - tangential_g;
7 float accelPitchCorrected = atan2(azCorrected,
8   sqrt(axCorrected * axCorrected + ay * ay)) * 180.0 / PI;
```

Despite the rigor, implementing and tuning it (correct axis assignment, rate smoothing to control derivative noise, several attempts at gating the correction’s influence) never fully resolved the symptom.

An intermediate workaround did make the system usable: reverting to a stripped-down control loop with zero correction terms and retuning the complementary filter so that pitch trusted its accelerometer more heavily than yaw (a 0.65/0.35 blend versus yaw’s 0.9/0.1). This worked well enough to collect a full set of results, but it was unsatisfying as an explanation. It directly contradicted the physics that had been pursued: centripetal and tangential acceleration are purely linear-acceleration effects, able to corrupt an accelerometer but with no physical mechanism to corrupt a rate gyroscope. If that were the whole story, the correct response would have been to trust pitch’s gyro *more*, not less, and yet trusting the accelerometer more was what empirically helped. That contradiction was a standing loose end.

The actual root cause

The true cause was eventually found, and it was far more mundane than any of the physics: a single sign error in the firmware. The pitch gyro rate was being read as

```
1 float gyroPitchRate = -(mpu.getGyroY() - GYRO_Y_OFFSET); // inverted
```

The leading negative sign inverted the gyroscope’s contribution to the complementary filter on the pitch axis. Because the filter’s gyro term is a dead-reckoning integration ($\theta \leftarrow \theta + \dot{\theta} \Delta t$), a sign-inverted rate meant that term integrated in the *wrong direction* on every loop, at every speed. This single line explains the entire cluster of symptoms cleanly: the prominent wrong-direction flick at the onset of fast tilts (a fast disturbance produces a large, rapidly growing gyro term, which if backward-signed drives the estimate the wrong way exactly at motion onset), and equally why trusting the accelerometer more appeared to help (down-weighting the broken gyro term partially masked the error, without ever fixing it). The accelerometer was never the more reliable signal; the gyro term was simply wrong on that axis.

Correcting the sign resolved everything. With the inversion removed, both axes run *fully identical* gains ($K_p = -1$, $K_i = -15$, $K_d = -0.0001$) and an identical 0.9/0.1 complementary filter blend, with no per-axis compensation of any kind. The pitch axis, previously the problem child requiring heavy accelerometer trust and settling roughly three times slower than yaw, now behaves essentially identically to it (see Section 10.6 for the before/after measurement).

The centripetal/tangential investigation is retained in this report rather than deleted, deliberately. The physics behind it is real and correctly derived, and pursuing it was a reasonable response to the evidence available at the time. But the episode is itself the most useful engineering lesson of the project: an elaborate, physically plausible model was built to explain a symptom whose real cause was a one-character firmware typo. The correction was ultimately found not by adding more sophistication, but by questioning why a workaround that contradicted the theory was working at all, and following that contradiction back to its source. Sometimes the most rigorous-looking explanation is not the correct one, and a standing contradiction between theory and observation is a signal worth chasing rather than tuning around.

8 WiFi Live-Tuning Dashboard

An asynchronous web server, hosted directly on the ESP32, exposes a lightweight dashboard for adjusting all six PID gains live, without reflashing, along with a polling telemetry readout of current angle and servo position. This proved essential for iterative tuning, since safety limits (the hard



Figure 15: Screenshot of the WiFi Live-Tuning Dashboard in the tuning process with detailed oscilloscope view showing raw outputs for az, ax, ay; corrected az; tangential term; alpha; gyro rate; pitch angle; and dt (s) from top to bottom.

15° clamp) remain enforced in firmware regardless of what values are entered. However, as with everything nice in this world, it did come with some tradeoffs. The live `/trace` endpoint builds a JSON string from several arrays via repeated string concatenation. This operation is apparently very taxing on the ESP32 architecture and polling this every $\sim 150\text{ms}$ while the dashboard is open steals enough loop time to visibly deteriorate the performance. This results in jumpy servo behavior that does not dissipate unless the dashboard is closed. In order to complete the results section under the system's highest performance, the dashboard was set to start recording for a preset amount of time (~ 20 seconds) before it could be closed and reopened after the test was complete.

9 Circuit Design & PCB

9.1 Switching Circuit Validation

The fan's PWM speed-control circuit (NPN transistor low-side switch) was validated in NI Multisim prior to physical construction, confirming correct base resistor sizing, transistor saturation behavior, and PWM waveform integrity via simulated oscilloscope measurement. Full simulation process notes are maintained separately in the project README.

9.2 PCB / Carrier Board Approach

Given that the majority of the system's electronics are pre-built breakout modules (ESP32, PCA9685, IMU) rather than raw components, the custom PCB was designed as a connector/header carrier board, providing mounting and wiring consolidation discrete fan-switching components, rather than a fully custom circuit. However, due to the limited project budget, the PCB was not physically implemented and the designed circuit was implemented into the breakout board, as seen in Figure 16.

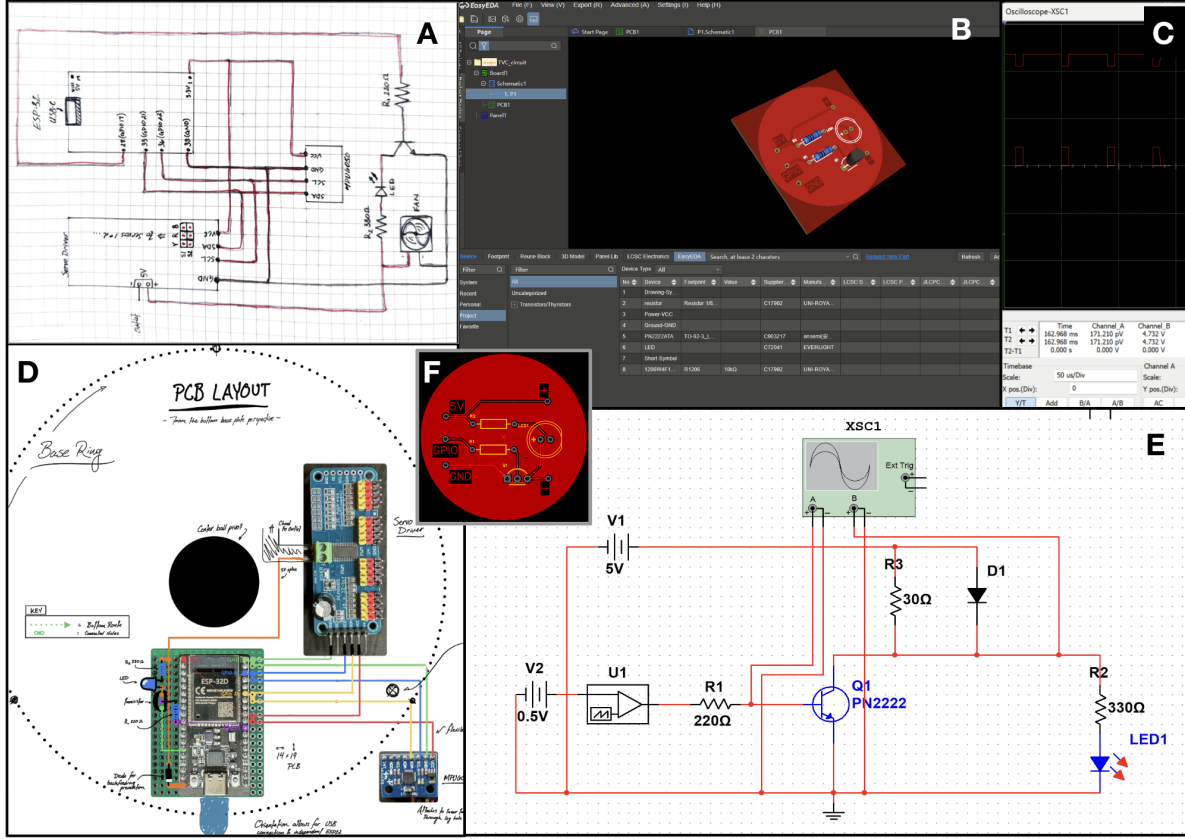


Figure 16: a) Schematic of full circuit from ESP32. b) 3D render of custom PCB created for switched fan integration on EasyEDA. c) Oscilloscope screenshot showing functional PWM output. d) Sketch of PCB layout mounted to the underside of the base plate (dotted circle) with 1:1 scale. e) Circuit diagram of simulated circuit for PWM fan powering - corresponding to part c which shows the output. f) 2-D rendering of custom PCB modeled in EasyEDA.

10 Results

Step-response data was captured directly from the running system via a recording buffer on the ESP32 (timestamped at the actual loop rate, ~ 24 ms per sample), triggered manually for each test session and downloaded as JSON for offline analysis. Three test sessions are presented, all under the final configuration (identical $K_p = -1$, $K_i = -15$, $K_d = -0.0001$ and 0.9/0.1 blend on both axes): a dedicated isolated-yaw session, a dedicated isolated-pitch session, and a combined session in which both axes were disturbed simultaneously. Each spans approximately 24 seconds and contains multiple distinct, manually-applied disturbances. Settling time is defined as the time from disturbance onset until the angle enters and remains within a $\pm 2^\circ$ band of its pre-disturbance baseline; steady-state error is the final angle relative to that baseline once settled.

10.1 Step Response: Isolated Axes



Figure 17: Isolated yaw axis: angle and servo output across 9 disturbances. All nine settled cleanly within the tolerance band.

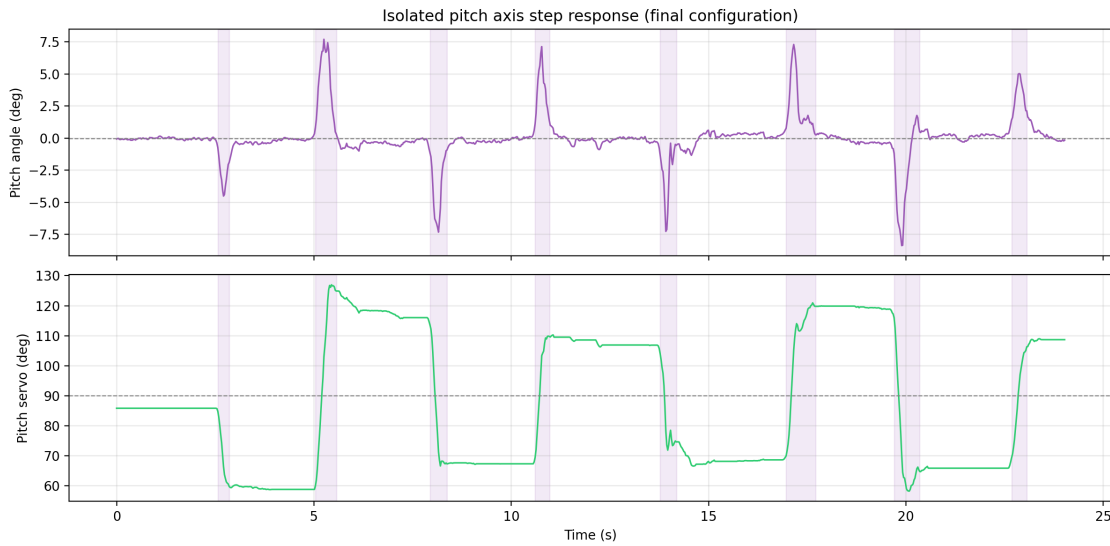


Figure 18: Isolated pitch axis: angle and servo output across 8 disturbances. All eight settled cleanly, at a mean settling time essentially identical to yaw's.

10.2 Step Response: Combined Two-Axis Disturbance

To confirm the axes remain well-behaved when driven together rather than in isolation, a final session disturbed both pitch and yaw simultaneously. Performance held up on both axes, with no cross-axis coupling or degradation relative to the isolated tests.

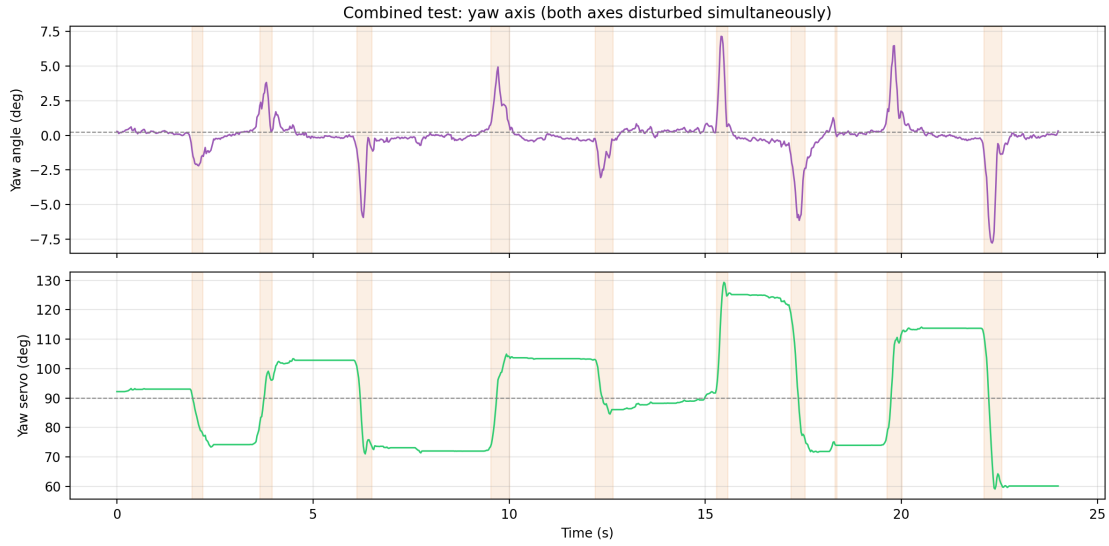


Figure 19: Combined test, yaw axis: 10 disturbances, all settled cleanly (mean 0.27) despite simultaneous pitch-axis activity.

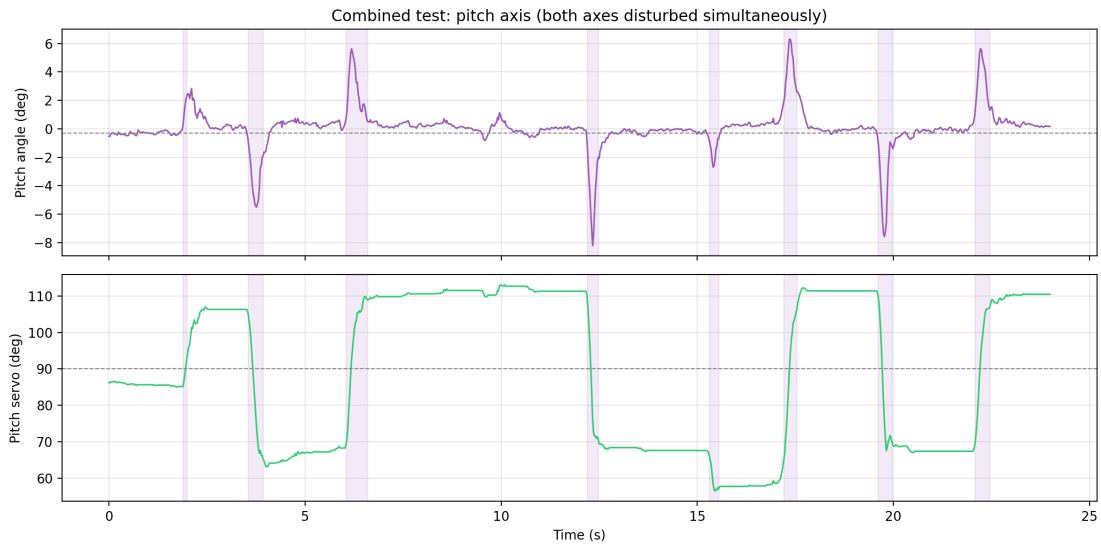


Figure 20: Combined test, pitch axis: 8 disturbances, 7 settled cleanly (mean 0.37) despite simultaneous yaw-axis activity.

10.3 Methodology and Worked Calculation

Four quantities are computed per disturbance event: peak deflection, true overshoot, settling time, and steady-state error. All depend on first establishing a baseline - the resting angle immediately before a given disturbance begins, computed as the mean of the angle signal over the quiet interval preceding it:

$$\beta = \frac{1}{n} \sum_{i=1}^n \theta_i, \quad i \in [\text{quiet interval}] \quad (55)$$

Worked example, yaw baseline. Using the 87 samples recorded before the first yaw disturbance:

$$\beta_{\text{yaw}} = \frac{1}{87} \sum_{i=1}^{87} \theta_i = 0.272^\circ \quad (56)$$

For each event, the search window runs from the disturbance's onset to the start of the *next* disturbance (or the end of the recording, for the final event). Peak deflection is the largest deviation from baseline anywhere in that window - this describes the size of the manually-applied disturbance itself, not the controller's response to it:

$$\theta_{\text{peak}} = \theta_i \text{ such that } |\theta_i - \beta| \text{ is maximized,} \quad i \in [\text{search window}] \quad (57)$$

Overshoot is distinct from peak deflection. Peak deflection measures how far the disturbance pushed the platform, which is a property of the applied disturbance, not the controller. True overshoot measures the controller's own behavior during correction: after the peak, does the angle cross back past the baseline and swing to the *opposite* side before settling, and if so by how much. This is computed by scanning the signal from the peak onward and taking the largest deviation on the opposite side of baseline from the disturbance itself:

$$\text{Overshoot} = \max_{i > i_{\text{peak}}} [-\text{sign}(\theta_{\text{peak}} - \beta) \cdot (\theta_i - \beta)] \quad (58)$$

taken with the sign of the crossing direction. An event that decays monotonically back to baseline without ever crossing to the opposite side has zero true overshoot.

Settling time is the first moment, measured from disturbance onset, after which the angle remains within a $\pm 2^\circ$ tolerance band around β for the remainder of the search window:

$$t_{\text{settle}} = t_j - t_{\text{onset}}, \quad j = \min\{i : |\theta_k - \beta| \leq 2^\circ \forall k \geq i \text{ within window}\} \quad (59)$$

Steady-state error is the angle at the end of the search window, relative to baseline:

$$e_{ss} = \theta_{\text{window end}} - \beta \quad (60)$$

Worked example, yaw event 2. Disturbance onset at $t = 4.848$ s; next disturbance begins at $t = 7.536$ s. Peak deflection occurs at $t = 5.016$ s, $\theta_{\text{peak}} = -11.93^\circ$ (disturbance size = $-11.93 - 0.27 = -12.20^\circ$, i.e. the size of the applied push - not the reported overshoot). Scanning forward from the peak, the angle crosses baseline and reaches a maximum of $\theta = 1.29^\circ$ before settling:

$$\text{Overshoot} = 1.29 - 0.27 = 1.01^\circ \quad (61)$$

The angle first enters and remains within the $\pm 2^\circ$ band at $t = 5.160$ s:

$$t_{\text{settle}} = 5.160 - 4.848 = 0.312 \text{ s} \quad (62)$$

At the end of the search window, $\theta = 0.762^\circ$:

$$e_{ss} = 0.762 - 0.272 = 0.490^\circ \quad (63)$$

Worked example, pitch event 7. Pitch’s largest recorded disturbance. Onset at $t = 19.700 \text{ s}$, baseline $\beta_{\text{pitch}} = -0.049^\circ$, next disturbance at $t = 22.676 \text{ s}$. Peak deflection at $t = 19.892 \text{ s}$, $\theta_{\text{peak}} = -8.38^\circ$ (disturbance size 8.33°). After the peak, the angle crosses baseline and reaches $\theta = 1.76^\circ$ before settling:

$$\text{Overshoot} = 1.76 - (-0.05) = 1.81^\circ \quad (64)$$

Settles at $t = 20.084 \text{ s}$:

$$t_{\text{settle}} = 20.084 - 19.700 = 0.384 \text{ s} \quad (65)$$

At window end, $\theta = 1.42^\circ$:

$$e_{ss} = 1.42 - (-0.05) = 1.47^\circ \quad (66)$$

This same procedure was applied identically to every recorded event across all three sessions; full per-event results for the isolated tests are given below.

10.4 Per-Event Results

Table 5: Isolated yaw axis, per-event results (9 events, all settled)

Event	Peak (deg)	Overshoot (deg)	Settling time (s)	Steady-state error (deg)
1	5.85	-1.03	0.26	-1.03
2	-12.20	1.01	0.31	0.49
3	6.05	-0.58	0.31	-0.58
4	-11.11	0.59	0.34	0.47
5	6.57	-1.18	0.38	-1.18
6	-7.34	0.47	0.41	0.23
7	5.54	-1.19	0.34	-1.14
8	-8.19	0.91	0.36	0.91
9	6.24	0.00	0.29	0.25

Table 6: Isolated pitch axis, per-event results (8 events, all settled)

Event	Peak (deg)	Overshoot (deg)	Settling time (s)	Steady-state error (deg)
1	-4.47	0.85	0.29	0.85
2	7.75	-1.11	0.46	-1.11
3	-7.28	1.17	0.34	1.17
4	7.18	-1.27	0.34	-1.27
5	-7.23	0.82	0.34	0.82
6	7.35	-1.14	0.34	-1.14
7	-8.33	1.81	0.38	1.47
8	5.08	-0.21	0.41	-0.11

10.5 Quantitative Summary

Table 7: Measured performance, final configuration (settled events only). Isolated columns are the dedicated single-axis sessions; combined columns are the simultaneous two-axis session.

Metric	Yaw (isolated)	Pitch (isolated)	Yaw (combined)	Pitch (combined)
Events settled	9/9	8/8	10/10	7/8
Settling time (mean)	0.33	0.36	0.27	0.37
Overshoot (mean)	0.77	1.05	0.90	0.58
Steady-state error (mean)	0.70	0.99	0.92	0.66

With the firmware sign error corrected and both axes running identical gains and blend ratios, the two axes now perform essentially the same. In the isolated tests, yaw settled cleanly on all 9 disturbances (mean 0.33) and pitch on all 8 (mean 0.36) - a gap of well under a tenth of a second, versus the roughly threefold gap seen before the fix (see Section 10.6). Mean overshoot stays near or below 1° on both axes despite disturbances of $6\text{--}12^\circ$, and steady-state error settles to within roughly $2\text{--}3\times$ the 0.35 deadband width on both, close to the practical precision floor of this control scheme given a generic IMU and hobby-grade servos.

The combined two-axis test confirms this holds up under simultaneous operation: with both axes disturbed at once, yaw settled all 10 disturbances (mean 0.27) and pitch 7 of 8 (mean 0.37), with overshoot and steady-state error comparable to the isolated cases. There is no meaningful cross-axis degradation - the mechanism steers in both axes at once without the axes interfering with one another, which is the behaviour a real thrust-vectoring application would require.

10.6 Before and After the Sign Correction

The single most consequential change in the entire project was the one-character firmware fix described in Section 7.5. Its effect on pitch-axis performance is best seen directly. Figure 21 shows the pitch response before the fix (running the heavy-accelerometer workaround, 0.65/0.35 blend, $K_p = -0.6$, $K_i = -6$) and after (identical configuration to yaw, 0.9/0.1 blend, $K_p = -1$, $K_i = -15$).

The improvement is not a tuning refinement but the removal of a defect: before the fix, pitch’s gyro term was integrating in the wrong direction every loop, and the heavy accelerometer blend was only masking it. After the fix, pitch needs no special treatment at all.

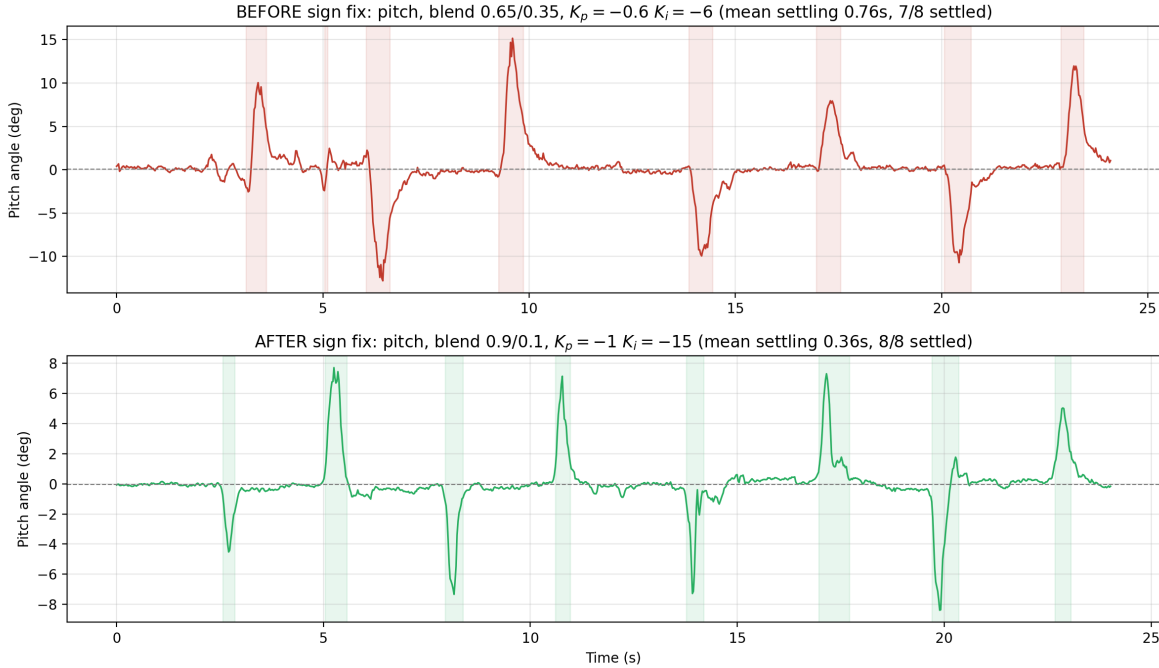


Figure 21: Pitch axis before (top) and after (bottom) the gyro sign correction. Before: mean settling 0.76, 7 of 8 disturbances settling cleanly, requiring a heavy accelerometer-trust workaround. After: mean settling 0.36, all 8 settling cleanly, running the same configuration as yaw. The correction roughly halved settling time and eliminated the axis asymmetry entirely.

11 Iteration & Lessons Learned

Throughout the project, there were many areas that were iterated upon. Namely, as mentioned previously in Section 4.4, the servo mounting brackets, the sheet metal design for the legs, and the stand/base. While there was significant physical iteration throughout this project, the majority was on the software side. An important lesson learned from debugging was a diagnostic-tooling observer effect, discovered through close observation of decreased servo performance while the WiFi dashboard was live. The fix, as previously mentioned, was to decouple recording from live serving and close the dashboard during sensitive or final tests. A second issue surfaced while implementing that fix: a heap-exhaustion bug caused by the firmware building the entire capture JSON as a single string via repeated concatenation before writing it to flash, which reliably crashed and produced truncated, corrupted capture files once the buffer was sized for a full-length session. The fix was to stream data directly to the file in small pieces rather than building one large string in memory first. A detailed, chronological debugging log (sensor calibration, PID tuning, firmware dependency issues) is maintained separately in the project README and referenced here rather than reproduced in full.

11.1 Named Future Improvements

- **Servo brackets:** current hand-cut brackets are functional but would benefit from 3D-printed replacements or thicker metal for improved rigidity, contingent on tool access.
- **Sensor mounting:** a more deliberate initial axis-alignment process (rather than iterative remount-and-recalibrate) would reduce setup time on any future build.
- **IMU position:** although the pitch/yaw asymmetry ultimately proved to be a firmware sign error rather than a genuine physical effect (Section 7.5), relocating the IMU to sit nearer the true geometric center, closer to equidistant from both pivots, would still be a worthwhile robustness improvement. The off-pivot mounting does produce real (if, as measured, non-dominant) motion-induced acceleration, and a centered mount would reduce it at the mechanical level, leaving the fused estimate cleaner regardless of the control tuning.
- **Tighter tolerances for pivots/linkages:** While the linkages worked remarkably well, the slight play caused by the linkage attachment hole being larger than the linkage pin introduced a small amount of backlash and reduced rigidity across the linkage connection. A potential fix would be to implement a new pin-and-hole geometry that permits free rotational motion while minimizing translational play.
- **Alternate servo mounting position:** While realistically inconsequential given the large torque margin already established in the torque budget, a possible improvement would be to mount the pitch-axis servo on the underside of the gimbal mechanism. The torque required to initially deflect the axis would remain roughly the same, but the gravitational moment present after a deflection would then act in the direction of the needed correction, rather than against it as in the current configuration - effectively making gravity assist the restoring torque instead of opposing it.

12 Conclusion

The production of this TVC demonstrator was initially meant to broaden my personal understanding of thrust vectoring and closed-loop control systems. By combining mechanical, electrical, controls, and software components, the goal was to develop a higher level of experience in each of these areas while creating a visually striking working apparatus that doubled as a test platform for further research into topics such as PID control, which is directly relevant to many other closed-loop systems.

The final demonstration achieved a sub-half-second mean settling time (0.33 s yaw, 0.36 s pitch) and mean overshoot near 1° (0.77° yaw, 1.05° pitch) - a strong result given the hand-fabrication methods involved. Reaching that point, however, required more than tuning: an asymmetry between the two axes persisted through both a rigorous physics-based investigation and simpler empirical fixes, and was only resolved once a direct contradiction between theory and observation was taken seriously rather than tuned around. That contradiction traced back to a single inverted sign on the pitch axis's gyroscope input - a one-line firmware error, not a physical effect at all. Correcting it brought both axes into full agreement with no further compensation needed.

There remain clear paths for improvement, particularly in structural rigidity - the servo brackets and general frame would need substantial reinforcement before this design could be considered for any real flight application. More broadly, this project sharpened my engineering judgment on a specific point: however elegant a proposed explanation may appear, it is essential to return to first principles and pursue rigorous root-cause analysis before accepting it.

13 Bill of Materials

Name	Material	Description	Mass	Qty	Part Number	Vendor
Servo 1	N/A	MG996R 55g digital metal gear servo	55 g	1	B07MFK266B	Amazon
Servo 2	N/A	MG996R 55g digital metal gear servo	55 g	1	B07MFK266B	Amazon
MPU6050 module	PCB - Fiber-glass	6-axis accelerometer/gyroscope, I2C, immersion gold PCB (generic, ICGOICIC)	1.8 g	1	ICGOICIC	AliExpress
Fan	ABS	3007 (30×30×7mm) 5V DC cooling fan	1.881 g	1	701715553996	Amazon
Fan Bracket	Aluminum 6061	Custom fab	2.553 g	1	—	—
Frame	PVC	Custom fab	82.711 g	1	—	—
Gimbal Leg	Aluminum	Custom fab	4.161 g	4	—	—
Inner Tube	PVC	Custom fab	59.597 g	1	—	—
Outer Ring	PVC	Custom fab	62.097 g	1	—	—
Screw, DIN 912 M3×8 A2	Hardened carbon steel	Socket head cap screw, A2 stainless	0.876 g	4	91292A112	McMaster-Carr
Servo1 Bracket	Aluminum 6061	Custom fab	4.58 g	1	—	—
Servo1 Linkage	Stainless steel	Custom fab	0.454 g	1	—	—
Servo2 Bracket	Aluminum 6061	Custom fab	2.621 g	1	—	—
Servo2 Linkage	Stainless steel	Custom fab	0.645 g	1	—	—
Base Plate	Basswood	Custom fab	304.92 g	1	—	—
Base Plate Disc	Basswood	Custom fab	105.887 g	1	—	—
Base Plate Disc*	Basswood	Custom fab	100.192 g	1	—	—
ESP32	PCB - Fiber-glass	WROOM-32D dev board, dual-core, WiFi+Bluetooth, USB-C	9 g	1	ESP32-DevKitC-32	Amazon
Ball Joint Pivot	N/A	Mini tripod ball head, 360° pivot	175.371 g	1	—	—
Servo Driver	PCB - Fiber-glass	PCA9685 16-channel 12-bit PWM driver, I2C	26.9 g	1	B07RMTN4NZ	Amazon

*Two Base Plate Disc entries with differing mass are present in the source CAD model; this likely reflects either two genuinely distinct disc parts or a stale duplicate from an earlier revision, and should be resolved directly in Onshape before this table is treated as final.

A Full Drawing Package

Drawing packages include revisions TVC-Rev-A through Rev-D. A full embeded revision history of the TVC engineering can be found in the TVC portfolio card found at this link: <https://cherrywookie.github.io/#card-tvc>.

B Firmware Listing

The final code logic is listed for quick reference below. This version excludes the WiFi dashboard and live-tuning features etc. For the full code with all features, please reference the GitHub repository at https://github.com/CherryWookiee/TVC_demonstrator/tree/main

```
1  // =====
2  // TVC Gimbal - Core Control Loop
3  // Sensor fusion, PID control, and servo output for the two-axis
4  // thrust vector control mechanism.
5  // =====
6
7  #include <Wire.h>
8  #include <Adafruit_PWMServoDriver.h>
9  #include <MPU6050_light.h>
10
11  Adafruit_PWMServoDriver pwm = Adafruit_PWMServoDriver(0x40);
12  MPU6050 mpu(Wire);
13
14  #define SDA_PIN 21
15  #define SCL_PIN 22
16  #define PITCH_CH 0
17  #define YAW_CH 1
18  #define SERVO_MIN 150
19  #define SERVO_MAX 600
20  #define SERVO_FREQ 50
21  #define SERVO_CENTER 90
22
23  #define LINKAGE_RATIO 2
24  #define GIMBAL_MAX_SAFE 20.0
25
26  #define FAN_PIN 17
27  #define FAN_PWM_FREQ 25000
28  #define FAN_PWM_RESOLUTION 8
29  #define FAN_SPEED 200
30
31  // --- Calibration (real measured values, final mounted position) ---
32  #define ACCEL_X_AVG -0.9616
33  #define ACCEL_Y_AVG -0.0555
34  #define ACCEL_Z_AVG -0.0584
35  #define GYRO_X_AVG 0.5768
36  #define GYRO_Y_AVG 1.3094
37  #define GYRO_Z_AVG -0.6862
38
39  // -X is up, so X_OFFSET = ax_avg - (-1.0), not ax_avg - 1.0
40  #define X_OFFSET (ACCEL_X_AVG + 1.0)
41  #define Y_OFFSET (ACCEL_Y_AVG)
42  #define Z_OFFSET (ACCEL_Z_AVG)
43  #define GYRO_X_OFFSET (GYRO_X_AVG)
44  #define GYRO_Y_OFFSET (GYRO_Y_AVG)
45  #define GYRO_Z_OFFSET (GYRO_Z_AVG)
46
47  // --- Final gains, identical on both axes ---
48  float Kp_pitch = -0.9;
49  float Ki_pitch = -15;
50  float Kd_pitch = -0.0001;
```

```

51
52 float Kp_yaw = -0.9;
53 float Ki_yaw = -15.0;
54 float Kd_yaw = -0.0001;
55
56 #define PITCH_DEADBAND 0.35
57 #define YAW_DEADBAND 0.35
58
59 #define INTEGRAL_LIMIT 60.0
60
61 float pitchAngle = 0, yawAngle = 0;
62 float pitchIntegral = 0, pitchLastError = 0;
63 float yawIntegral = 0, yawLastError = 0;
64 bool pitchSaturated = false, yawSaturated = false;
65 unsigned long lastTime = 0;
66
67 int angleToPulse(float angle) {
68     return (int)((angle) * (SERVO_MAX - SERVO_MIN) / 180.0 + SERVO_MIN);
69 }
70
71 // Soft deadband: below this error magnitude, contribute nothing. Above
72 // it, the error ramps up continuously from zero rather than jumping to
73 // its full raw value - no discontinuity at the boundary.
74 float applyDeadband(float error, float deadband) {
75     if (fabs(error) <= deadband) return 0.0;
76     return error - deadband * (error > 0 ? 1.0 : -1.0);
77 }
78
79 float computePID(float setpoint, float measured, float &integral,
80                 float &lastError, float dt, float Kp, float Ki, float Kd,
81                 bool outputSaturated, float deadband) {
82     float rawError = setpoint - measured;
83     float error = applyDeadband(rawError, deadband);
84
85     // Anti-windup: only accumulate integral if the output ISN'T already
86     // saturated, or if this error would pull the output back INTO range.
87     bool wouldIncreaseSaturation = (outputSaturated && ((error > 0) == (integral >
88         0)));
89     if (!wouldIncreaseSaturation) {
90         integral += error * dt;
91     }
92     integral = constrain(integral, -INTEGRAL_LIMIT, INTEGRAL_LIMIT);
93
94     float derivative = (dt > 0) ? (error - lastError) / dt : 0;
95     lastError = error;
96     return Kp * error + Ki * integral + Kd * derivative;
97 }
98
99 void setup() {
100     Serial.begin(115200);
101     Serial.println("===TVC_Gimbal_core_control_loop===");
102
103     Wire.begin(SDA_PIN, SCL_PIN);
104
105     pwm.begin();
106     pwm.setPWMPFreq(SERVO_FREQ);
107     delay(10);
108     pwm.setPWM(PITCH_CH, 0, angleToPulse(SERVO_CENTER));
109     pwm.setPWM(YAW_CH, 0, angleToPulse(SERVO_CENTER));

```

```

109 Serial.println("Servos centered.");
110
111 ledcSetup(1, FAN_PWM_FREQ, FAN_PWM_RESOLUTION);
112 ledcAttachPin(FAN_PIN, 1);
113 ledcWrite(1, FAN_SPEED);
114
115 byte status = mpu.begin();
116 Serial.print("MPU6050 status: "); Serial.println(status);
117 if (status != 0) {
118     Serial.println("MPU6050 not found!");
119     while (1) { delay(10); }
120 }
121
122 mpu.fetchData();
123 float ax = mpu.getAccX() - X_OFFSET;
124 float ay = mpu.getAccY() - Y_OFFSET;
125 float az = mpu.getAccZ() - Z_OFFSET;
126 pitchAngle = atan2(az, sqrt(ax * ax + ay * ay)) * 180.0 / PI;
127 yawAngle = atan2(ay, sqrt(ax * ax + az * az)) * 180.0 / PI;
128
129 Serial.println("Setup complete. Stabilization active.");
130 delay(1000);
131 lastTime = millis();
132 }
133
134 void loop() {
135     mpu.fetchData();
136
137     float ax = mpu.getAccX() - X_OFFSET;
138     float ay = mpu.getAccY() - Y_OFFSET;
139     float az = mpu.getAccZ() - Z_OFFSET;
140
141     float accelPitch = atan2(az, sqrt(ax * ax + ay * ay)) * 180.0 / PI;
142     float accelYaw = atan2(ay, sqrt(ax * ax + az * az)) * 180.0 / PI;
143
144     unsigned long now = millis();
145     float dt = (now - lastTime) / 1000.0;
146     lastTime = now;
147
148     // Pitch gyro rate is negated - corrects a sign inversion in the raw
149     // sensor reading relative to the axis convention used here.
150     float gyroPitchRate = -(mpu.getGyroY() - GYRO_Y_OFFSET);
151     float gyroYawRate = mpu.getGyroZ() - GYRO_Z_OFFSET;
152
153     // Complementary filter: identical 0.9/0.1 blend on both axes.
154     pitchAngle = 0.9 * (pitchAngle + gyroPitchRate * dt) + 0.1 * accelPitch;
155     yawAngle = 0.9 * (yawAngle + gyroYawRate * dt) + 0.1 * accelYaw;
156
157     float pitchCorrection = computePID(0, pitchAngle, pitchIntegral,
158                                         pitchLastError, dt,
159                                         Kp_pitch, Ki_pitch, Kd_pitch, pitchSaturated
160                                         ,
161                                         PITCH_DEADBAND);
162     float yawCorrection = computePID(0, yawAngle, yawIntegral,
163                                         yawLastError, dt,
164                                         Kp_yaw, Ki_yaw, Kd_yaw, yawSaturated,
165                                         YAW_DEADBAND);
166
167     // Determine saturation BEFORE clamping, so next loop's anti-windup

```

